

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 19: SIMULANDO TRANSFERÊNCIA DE CALOR EM GPU COM OPENMP

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 25/09/2026.

1. EXPERIMENTO

Faça o exercício de transferência de calor (heat.c, slide 64) do tutorial de programação de GPUs com OpenMP. Explore as diretivas de paralelização e movimentação de dados para copiar dados entre host e dispositivo. Experimente com diferentes tamanhos de problema perfilando o programa com o nsys.

2. CONCEITOS RELEVANTES

- **Stencil de cinco pontos:** calcula a nova temperatura de cada célula usando seu valor atual e os valores dos quatro vizinhos. Cada célula pode ser atualizada em paralelo dentro de um mesmo passo.
- **Campos de entrada e saída:** “u” armazena as temperaturas atuais e “u_tmp” recebe as novas. Essa separação evita sobrescrever valores necessários ao cálculo das células vizinhas.
- **target teams loop:** encaminha o cálculo à GPU e distribui as iterações entre equipes e threads. A cláusula device(0) seleciona o dispositivo de índice zero.

- **collapse(2):** associa os dois laços da malha à distribuição do trabalho, permitindo paralelizar as "n²" atualizações.
- **Mapeamento e transferência de dados:** "map(to: u[0:n*n])" envia o campo de entrada à GPU, enquanto "map(from: u_tmp[0:n*n])" devolve o resultado à CPU. Como essas cláusulas estão dentro de solve, as transferências ocorrem a cada passo.
- **Sincronização e troca de ponteiros:** a região na GPU termina antes da troca dos ponteiros "u" e "u_tmp". Assim, o resultado torna-se a entrada do próximo passo sem copiar os elementos entre os arrays.
- **Medição de desempenho:** "omp_get_wtime()" mede o tempo do laço temporal. O experimento calcula a mediana de três execuções sem perfilamento e utiliza uma execução separada com "nsys" para registrar os tempos dos kernels e das cópias.

3. MÁQUINA UTILIZADA

No NPAD, para realizar os testes relacionados à tarefa, foi utilizada a partição *gpu-8-v100*.

4. CÓDIGO

Figura 1: Código do experimento

```
#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include <omp.h>

// Constantes usadas no cálculo e na apresentação dos resultados.
#define PI acos(-1.0) // Valor de pi.
#define LINE "--------------------\n" // Separador da saída.

// Protótipos das funções. Os campos são arrays lineares de n*n doubles.
// restrict indica que os dados acessados por esses ponteiros não se sobrepõem.
```

```c
void initial_value(const int n, const double dx, const double length, double * restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double dt, const
double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const double alpha, const
double length);
double l2norm(const int n, const double * restrict u, const int nsteps, const double dt,
const double alpha, const double dx, const double length);

// Configura o problema, executa os passos e verifica a solução na CPU.
int main(int argc, char *argv[]) {

 // Tempo total: inclui configuração, inicialização, solve e norma do erro.
 // omp_get_wtime retorna o tempo de parede em segundos.
 double start = omp_get_wtime();

 // Valores padrão: n células internas em cada direção e nsteps passos.
 int n = 1000;

 // O intervalo físico total permanece 0,5 ao variar o número de passos.
 int nsteps = 10;

 //
 // Parâmetros físicos e discretização.
 //
 double alpha = 0.1;         // Coeficiente de difusão térmica.
 double length = 1000.0;     // Lado do domínio quadrado.
 double dx = length / (n+1);  // Espaçamento entre pontos internos e bordas.
 double dt = 0.5 / nsteps;    // Passo temporal: tempo físico total de 0,5.



 // r é o coeficiente adimensional da discretização explícita.
 // Para este stencil 2D com espaçamentos iguais, a estabilidade exige r <= 1/4.
 double r = alpha * dt / (dx * dx);

 // Registra os parâmetros utilizados para permitir interpretar os resultados.
 printf("\n");
 printf(" MMS heat equation\n\n");
 printf(LINE);
 printf("Problem input\n\n");
```

```
printf(" Grid size: %d x %d\n", n, n);
printf(" Cell width: %E\n", dx);
printf(" Grid length: %lf x %lf\n", length, length);
printf("\n");
printf(" Alpha: %E\n", alpha);
printf("\n");
printf(" Steps: %d\n", nsteps);
printf(" Total time: %E\n", dt*(double)nsteps);
printf(" Time step: %E\n", dt);
printf(LINE);

// Aviso herdado do código base: usa 0,5, embora o limite 2D seja 0,25.
// Nas malhas do experimento (até 4096, com 100 passos), r fica abaixo de 0,25.
printf("Stability\n\n");
printf(" r value: %lf\n", r);
if (r > 0.5)
  printf(" Warning: unstable\n");
printf(LINE);



// Aloca os dois campos na memória da CPU. A posição (i,j) fica em i+j*n.
// u é a entrada do passo; u_tmp recebe a saída, sem sobrescrever a entrada.
double *u     = malloc(sizeof(double)*n*n);
double *u_tmp = malloc(sizeof(double)*n*n);
double *tmp;

// Inicializa o campo senoidal e zera o segundo array na CPU.
initial_value(n, dx, length, u);
zero(n, u_tmp);

//
// Evolução temporal pelo esquema explícito.
//

// Mede somente o laço temporal: inclui cópias, kernels, sincronização
// e custos do runtime, inclusive os da primeira chamada ao dispositivo.
double tic = omp_get_wtime();
for (int t = 0; t < nsteps; ++t) {

  // Calcula o próximo estado na GPU e devolve u_tmp à CPU.
```

```
  // Os passos são sequenciais: cada um depende do estado anterior.
  solve(n, alpha, dx, dt, u, u_tmp);

  // Troca apenas os endereços: o resultado passa a ser a próxima entrada.
  // Não há cópia dos elementos entre os arrays nessa operação.
  tmp = u;
  u = u_tmp;
  u_tmp = tmp;
 }
 // A verificação numérica abaixo fica fora do tempo de solve.
 double toc = omp_get_wtime();

 //
 // Compara o campo final com a solução analítica da MMS.
 //
 double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

 // Encerra a medição total após a verificação da solução.
 double stop = omp_get_wtime();

 // Imprime o erro e os dois tempos, extraídos pelo script do experimento.
 printf("Results\n\n");
 printf("Error (L2norm): %E\n", norm);
 printf("Solve time (s): %lf\n", toc-tic);
 printf("Total time (s): %lf\n", stop-start);
 printf(LINE);

 // Libera ambas as alocações, independentemente das trocas de ponteiros.
 free(u);
 free(u_tmp);

}



// Condição inicial senoidal da solução manufaturada (MMS).
// Os pontos internos começam em dx; as bordas externas têm temperatura zero.
void initial_value(const int n, const double dx, const double length, double * restrict u) {

 double y = dx;
 for (int j = 0; j < n; ++j) {
```

```
    double x = dx; // Coordenada física da primeira célula interna da linha.
    for (int i = 0; i < n; ++i) {
      u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
      x += dx;
    }
    y += dx; // Avança para a próxima linha da malha.
  }

}



// Preenche o array com zeros na CPU antes do laço temporal.
void zero(const int n, double * restrict u) {

  for (int i = 0; i < n; ++i) {
    for (int j = 0; j < n; ++j) {
      u[i+j*n] = 0.0;
    }
  }

}



// Atualiza um passo pelo método explícito de diferenças finitas.
// A entrada é somente leitura; cada iteração escreve uma célula da saída.
void solve(const int n, const double alpha, const double dx, const double dt, const
double * restrict u, double * restrict u_tmp) {

  // Peso dos vizinhos: r; peso da célula central: 1 - 4*r.
  const double r = alpha * dt / (dx * dx);
  const double r2 = 1.0 - 4.0*r;

  // target executa a região no dispositivo selecionado por device(0).
  // teams loop distribui o trabalho entre equipes e threads da GPU.
  // collapse(2) associa os dois laços à distribuição das n*n células.
  // map(to) envia a entrada; map(from) devolve a saída, sem enviá-la antes.
  // [0:n*n] especifica todos os elementos dos campos apontados pelos ponteiros.
  // Sem uma região externa de dados persistentes, as cópias se repetem
  // em cada chamada: um envio e um retorno por passo temporal.
  // Sem nowait, a região termina e a saída retorna antes da troca na CPU.
```

```
 #pragma omp target teams loop device(0) collapse(2) map(to : u[0:n*n]) map(from :
u_tmp[0:n*n])
 for (int i = 0; i < n; ++i) {
  for (int j = 0; j < n; ++j) {

      // Stencil: centro e vizinhos nas direções +i, -i, +j e -j.
      // Os testes impedem acesso fora do array e aplicam as bordas nulas.
      // A ordem original mantém j no laço interno: o índice avança n posições.
      u_tmp[i+j*n] =  r2 * u[i+j*n] +
      r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
      r * ((i > 0)   ? u[i-1+j*n] : 0.0) +
      r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
      r * ((j > 0)   ? u[i+(j-1)*n] : 0.0);
  }
 }
}


// Solução analítica: o perfil senoidal decai exponencialmente com o tempo.
// Serve como referência para verificar o campo calculado numericamente.
double solution(const double t, const double x, const double y, const double alpha, const
double length) {

 return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) * sin(PI*y/length);

}


// Calcula na CPU a norma euclidiana do erro: sqrt(soma((u - solução)^2)).
// Não normaliza pelo número de células nem pondera pela área de cada célula.
double l2norm(const int n, const double * restrict u, const int nsteps, const double dt,
const double alpha, const double dx, const double length) {

 // Instante físico final, igual a 0,5 para os parâmetros deste programa.
 double time = dt * (double)nsteps;

 // Acumula os quadrados das diferenças em todas as células internas.
 double l2norm = 0.0;

 // Percorre o campo e avalia a referência nas mesmas coordenadas físicas.
```

```
  double y = dx;
  for (int j = 0; j < n; ++j) {
    double x = dx;
    for (int i = 0; i < n; ++i) {
      double answer = solution(time, x, y, alpha, length);
      l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);

      x += dx;
    }
    y += dx;
  }

  return sqrt(l2norm);

}
```

5. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho do experimento.

Figura 2: Resultado usando paralelização com CPU

Malha	Mediana de solve (s)	Kernels (ms)	Cópias (ms)
256 × 256	0,270974	0,775	8,861
512 × 512	0,328433	2,26	38,761
1024 × 1024	0,515677	7,111	236,551
2048 × 2048	1,176902	27,5	897,092
4096 × 4096	3,774509	109,959	3513,14

**O tempo de solve é a mediana de três execuções sem perfilamento. Os tempos acumulados dos kernels e das cópias foram obtidos em uma execução separada com "nsys".*

Ao analisar os dados dos resultados, pode-se perceber que as cópias levaram mais tempo que os kernels porque, nesta implementação, os campos são transferidos em todos os passos. Antes do cálculo, "map(to)" envia as temperaturas de "u" à GPU. Depois, "map(from)" devolve as temperaturas calculadas em "u_tmp" à CPU. Para 100 passos, esse fluxo se repete 100 vezes.

Na malha 4096 × 4096, cada campo ocupa 12 MiB, pois contém 4096^2 elementos de 8 bytes. Enviar um campo e devolver o outro movimenta 256 MiB por passo, totalizando 25 GiB nos 100 passos. Esse grande volume ajuda a explicar os 3513,140ms acumulados nas cópias, frente aos 109,959ms dos kernels. A GPU calcula o stencil rapidamente, mas precisa receber e devolver os campos repetidamente.

Nas malhas menores, o tempo de solução não acompanha tão diretamente o número de células. Isso acontece porque a execução também envolve custos de preparação do dispositivo, gerenciamento dos mapeamentos, lançamento dos kernels e sincronização. Esses custos não diminuem necessariamente na mesma proporção que a malha. Assim, quando o cálculo é pequeno, eles representam uma parcela maior do tempo medido.

Por fim, o tempo de solução inclui todo o laço temporal, enquanto o perfilamento registra separadamente as atividades da GPU. Como essas medições também vêm de execuções distintas, a soma dos tempos de kernels e cópias não precisa coincidir com a mediana de “solve”. A principal conclusão dos resultados é que, nesta versão, a movimentação repetida dos campos custa muito mais que o cálculo do stencil.